

Partiel

Ghiles Ziat

Déroulement de L'examen

Tous les documents sont autorisés, et notamment ceux du cours.

L'utilisation de tout moyen de communication (téléphone, tablette, ordinateur personnel, etc.) est interdit.

L'utilisation d'une clé ou disque USB personnel, en lecture seule et étiqueté à votre nom, est autorisée; vous ne devez pas le prêter à un camarade.

Le barème pour chaque question n'est donné qu'à titre indicatif et pourra être adapté lors de la correction.

L'examen dure 2h.

Votre copie sera formée de fichiers textuels que vous laisserez aux endroits spécifiés dans votre espace de travail pour Eclipse. L'espace de travail pour Eclipse sera obligatoirement nommé `workspace` et devra être un sous-répertoire direct de votre répertoire personnel.

Le langage à étendre est ILP2. Le paquetage Java correspondant sera nommé `com.paracamplus.ilp2.partiel1718.ilp2switch`. Sera ramassé, à partir de votre `~/workspace` (situé sous ce nom directement dans votre HOME), le **seul** répertoire `~/workspace/ILP-UPMC/Java/src/com/paracamplus/ilp2/partiel1718/ilp2switch` et tous ses sous-répertoires.

Pour vous éviter de la taper à nouveau, voici l'URL du site du Master (mais il faut, pour y accéder, ne plus passer par le proxy) et celle de l'ARI où se trouvent de nombreuses documentations dont celle de Java :

<http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/>
<http://www-ari.ufr-info-p6.jussieu.fr/>

L'examen sera corrigé à la main, il est donc absolument inutile de s'acharner sur un problème de compilation ou sur des méthodes à contenu informatif faible. Il est beaucoup plus important de rendre aisé, voire plaisant, le travail du correcteur et de lui indiquer, par tout moyen à votre convenance, de manière claire, compréhensible et terminologiquement précise, comment vous surmontez cette épreuve.

Mise en route

- Ouvrez un navigateur et allez à l'URL : <https://www-master.ufr-info-p6.jussieu.fr:8083/2017/DLP>. Vous y trouverez les sources d'ILP ainsi qu'une version PDF de ce sujet.
- Ouvrez Eclipse.
- Choisissez le répertoire `workspace` comme `workspace`.
- Extrayez l'archive dans le répertoire `workspace` (vous devez obtenir le répertoire `~/workspace/ILP-UPMC`).

- Créez un nouveau projet Java en cliquant sur : File -> New -> Java Project.
- Choisissez ILP-UPMC en "project name". En bas de la fenêtre devrait s'afficher le message : "the wizard will automatically. . .", indiquant qu'Eclipse a détecté qu'un projet de ce nom est déjà présent.
- Cliquez sur Next.
- (facultatif, puisqu'on ne demande pas le code du compilateur) Compilez la bibliothèque d'exécution en faisant :

```
cd ~/workspace/ILP-UPMC/C/; make
```

- Le plug-in ANTLR devrait fonctionner sans réglages particuliers. Toutefois, si ce n'est pas le cas, vous pouvez toujours utiliser ANTLR en ligne de commande à l'aide de l'archive fournie dans les sources d'ILP, en faisant :

```
cd ~/workspace/ILP-UPMC/ANTLRGrammars
java -jar ~/workspace/ILP-UPMC/Java/jars/antlr-4.4-complete.jar \
  -lib ~/workspace/ILP-UPMC/ANTLRGrammars \
  -o ~/workspace/ILP-UPMC/target/generated-sources/antlr4/ \
  LE_NOM_DE_VOTRE_GRAMMAIRE.g4
```

Rappelons qu'il est aussi possible de lancer le plug-in à la main depuis Eclipse avec un clic droit sur la grammaire puis "Run as. . ." si il ne se lance pas automatiquement lors de l'édition d'une grammaire.

Troubleshooting

Si Eclipse se bloque en essayant d'installer un connecteur SVN, tapez dans un terminal :

```
killall eclipse; killall java
```

puis relancez Eclipse.

Introduction

Pour cet examen, nous nous proposons d'étendre ILP2 et d'y inclure la construction `switch`. L'instruction `switch` permet de faire plusieurs tests sur la valeur d'une même expression.

Sa syntaxe est illustrée dans l'exemple suivant :

```
switch e1 do
  case e2 : e3;
  case e4 : e5;
  otherwise : e6;
done
```

Où `e1`, `e2`, `e3`, `e4`, `e5` et `e6` sont des expressions arbitraires de notre langage et **"switch"**, **"do"**, **"case"**, **"otherwise"** et **"done"** des mots-clés de l'expression `switch`.

Ce branchement conditionnel peut aussi être considéré comme équivalent à une imbrication d'alternatives (expressions `if`). Par exemple, le programme suivant est équivalent au premier exemple :

```
let x = e1 in
if x == e2 then e3
else if x == e4 then e5
else e6
```

Votre implantation devra respecter les conditions suivantes :

- Vous devez permettre un nombre arbitraire d'instructions `case`. Chaque cas est suivi de la valeur à comparer, d'un caractère ":", d'une expression à évaluer si la comparaison a réussi et est terminé par le caractère ";".
- Une instruction `switch` peut avoir un cas par défaut facultatif dénoté par le mot-clé **otherwise**, qui doit apparaître en dernière instruction du `switch`. Ce cas par défaut doit être utilisé pour effectuer un traitement lorsqu'aucune des comparaisons avec les instructions `case` n'a réussi.
- La valeur retournée par un `switch` devra être la valeur de l'expression suivant le premier **case** dont la comparaison avec la valeur testée a réussi.
- Lorsqu'aucune valeur suivant un **case** n'est égale à la valeur testée, alors la valeur retournée par le `switch` est celle de l'expression par défaut suivant le **otherwise**, si il y en a une.
- Lorsqu'aucune valeur suivant un **case** n'est égale à la valeur testée, et qu'il n'y a pas d'instruction **otherwise** alors la valeur de retour du `switch` est `false`.
- La valeur testée (l'expression `e1` dans l'exemple) par le `switch` ne devra être évaluée qu'une seule fois (donc l'exemple ci-dessus n'est pas équivalent à `if e1==e2 then e3 else if e1==e4 then e5 else e6`, car l'expression `e1` peut être évaluée une deuxième fois si le premier test échoue, ce que nous interdisons).
- La fonction de comparaison utilisée par votre implantation devra être la même que celle utilisée par l'opérateur `==` d'ILP.
- Vous ferez bien attention à travailler avec des expressions et non pas des constantes, que ce soit pour la valeur testée ou les différents cas avec lesquels on la compare. Par exemple le programme suivant est censé être accepté par votre grammaire.

```
function carre (x)
  x*x;

function mention (note)
  let x=2 in
  switch (note+note)/2 do
    case 5          : print( "Très bien" );
    case carre(2)   : print( "Bien" );
    case 2+1        : print( "Assez bien" );
    case x          : print( "Passable" );
    case 1.0        : print( "Echec" );
    otherwise       : print( "Note non reconnue" )
  done;

x = 1;
while x <= 6 do (
  note(x);
  x = x+1
)
```

Ce programme retournera `false` et affichera sur la sortie standard

```
EchecPassableAssez bienBienTrès bienNote non reconnue
```

Attention : En ILP2, la comparaison n'est définie que sur les valeurs numériques. Ne soyez pas surpris si la comparaison de valeurs non-numériques (chaînes de caractères par exemple) provoque une erreur d'exécution.

1 Grammaire (2 points)

Écrivez une grammaire définissant ce nouveau dialecte nommée `com.paracamplus.ilp2.partiel1718.ilp2switch.parser.ilp2switch.g4`.

2 AST (3 points)

Proposez une extension de l'arbre syntaxique avec la nouvelle construction `switch`. Rappelons que vous devez fournir les classes AST ainsi que les interfaces correspondantes (resp. dans `com.paracamplus.ilp2.partiel1718.ilp2switch.ast` et `com.paracamplus.ilp2.partiel1718.ilp2switch.interfaces`). N'oubliez pas d'inclure la classe `ASTfactory` correspondante et son interface `IASTfactory` ainsi que la nouvelle interface `IASTvisitor`.

3 Analyse syntaxique (2 points)

Écrivez dans le paquetage `com.paracamplus.ilp2.partiel1718.ilp2switch.parser` les nouvelles classes `ILPMLparser` et `ILPMLlistener` permettant de construire les nouveaux noeuds ajoutés au langage.

4 Interprétation (8 points)

Rappelons que nous avons vu en cours deux façons d'intégrer la gestion de nouveaux noeuds à un interprète ou un compilateur :

- Conserver le nouveau noeud jusqu'à l'interprétation et y intégrer son traitement dans une nouvelle classe `Interpreter`.
- Transformer avant l'interprétation les occurrences des nouveaux noeuds en un ou plusieurs noeuds déjà présents dans le langage, qui produisent le même comportement. Cette méthode offre l'avantage de pouvoir réutiliser l'interprète et le compilateur sans les modifier.

Vous écrirez les deux méthodes dans la suite, en faisant bien attention à produire un comportement similaire pour les deux implantations.

4.1 Interprète (4 points)

Implantez le traitement des nouveaux noeuds dans une classe `com.paracamplus.ilp2.partiel1718.interpreter.Interpreter`.

4.2 Réécriture (4 points)

Comme souligné dans l'introduction, remarquons que les deux programmes suivants sont équivalents :

```
—
switch e1 do
  case e2 : e3;
  case e4 : e5;
  otherwise : e6
done
```

```

—
let x = e1 in
if x == e2 then e3
else if x == e4 then e5
else e6

```

Remarquons de plus que le résultat de l'évaluation de `e1` est stocké dans une variable qui est utilisée ensuite dans l'imbrication d'alternatives. Ainsi, l'expression `e1` n'est bien évaluée qu'une seule fois.

Implantez cette transformation dans une classe

`com.paracamplus.ilp2.partiel1718.interpreter.ASTtransformSwitch`

Vous n'êtes pas obligés de produire un code complet : vous pourrez détailler uniquement les noeuds AST pertinents pour cette transformation, et expliquer succinctement en commentaire dans le code ce qui reste à faire pour obtenir une transformation d'AST complète.

5 Compilation (5 points)

5.1 Questions (2.5 points)

Répondez aux questions suivantes :

- listez les classes et interfaces que vous auriez à ajouter pour mettre en œuvre le compilateur, en précisant pour chacune les classes et interfaces qu'elles étendent ou implantent ;
- listez les fonctions existantes de la bibliothèque d'exécution C que vous exploitez si vous en exploitez, ce qu'elles font, et comment vous les utilisez ; listez également les ajouts éventuels que vous devez apporter à la bibliothèque d'exécution C ;
- précisez les erreurs d'exécution potentielles que l'implantation doit prendre en compte ;
- précisez quels aspects de l'instruction `switch` et de son exécution sont statiques, et lesquels sont dynamiques.

5.2 Schéma de compilation (2.5 points)

Proposez un schéma de compilation pour les instructions `switch`. Il ne sera pas nécessaire de fournir le code correspondant.

6 Bonus : Tests (2 points)

Créez un répertoire

`com/paracamplus/ilp2/partiel1718/ilp2switch/SamplesILP2switch`

dans lequel vous mettrez un programme ILP illustrant un exemple intéressant d'utilisation du `switch`. Vous fournirez bien sûr les 3 fichiers : `.ilpml`, `.print` et `.result`. Testez votre implantation sur votre programme en fournissant la classe de test

`com.paracamplus.ilp2.partiel1718.interpreter.test.InterpreterTest`.

Elle devra être compatible avec les précédentes versions du langage.